BibMerge — Principles & Practices

CS5013 Programming with AI · Mid-evaluation deliverable · S Naveen (CS26E010) and Gowni Vamshi (CS26E005) · github.com/Sajjan-Naveen-87/BibMerge · 11 Oct 2026

BibMerge merges the `.bib` files of several papers into one bibliography, recognising the same work under different keys and formatting. The engine is plain Java 21 (9 pipeline stages), served by a Spring Boot API and CLI. Every practice below is something we actually did; the evidence is in the repository: 5 merged PRs, 26 commits by both members, 164 tests, and the prompt log in `doc/prompts/`.

1. Prompt engineering

Our prompts improved most when we stopped asking for features and started giving the model the same inputs a teammate would get. Five principles, each learned from a prompt that failed first:

- **Context before task.** Point the model at the design-doc section, the existing interfaces and the current repository state before asking for code (P1, P2). A handoff document said the design doc existed; the repository audit showed it had never been pushed.
- **One unit of work per prompt.** One pipeline stage or one bug per prompt, one commit per answer. Large prompts produced code we could not review.
- **Constraints and non-goals stated explicitly.** Language, package, interfaces that must not change, and what not to touch: "do not write Vamshi's modules" (P3) kept ownership honest.
- **Executable acceptance criteria.** Every prompt names the tests that must pass and the command to run; we ask for test counts back, not "done".
- **Make the model verify against reality.** "Run it on the stakeholder's real papers and explain every decision" (P4) found a bug no fixture had.

Template we now use: *Context* → *Task* → *Constraints* → *Acceptance* → *Report* (Appendix). Each logged prompt names the commit it produced.

2. Debugging

Our loop is **reproduce** → **shrink to a failing test** → **hypothesise** → **fix** → **full suite** → **keep the test**. We never fix a bug we cannot reproduce in a test, and every fix lands with its regression test in the same commit.

Symptom (how found)	Root cause	Fix + guard
One unclosed brace lost every later entry (parser probe)	JBibTeX consumes to EOF on unbalanced braces	Parse per entry chunk; test *malformedEntryLosesOnlyItself*
Accented names lost their accents: Erd\H{o}s became "Erdos" (golden table test)	Library LaTeX printer silently drops \H and \i	Own decoder; 17-case parameterised table
"DOI: 10.1109/…" produced no DOI (table test)	Value cut at first space before prefix strip	Reordered; 16 DOI variants tested
Part I vs Part II scored 0.976 (printed every pair's score)	Title similarity cannot see part numbers	Part-number veto + synthetic corpus case
Two different ePrint papers sent to review (real data)	ePrint record carried its published DOI, bypassing the id veto	Different preprint ids always veto; 4 tests
\cite{oldkey} could resolve to another paper (edge probe)	Minted key equal to another entry's retired key	Block other entries' keys; pipeline test

Two process lessons: print the evidence (scores, rule fired, features) for every decision rather than only pass/fail, which is how the Part I/II bug surfaced; and probe module *boundaries* after integration, because each module passed its own tests while the key collision only existed between clustering and golden-record building.

3. Version control (Git) for code and prompts

- **Branch per unit, PR per branch, CI on every PR.** GitHub Actions runs `mvn verify` and the frontend lint/build; PRs merge only on green, with merge commits so each member's history stays visible (`git shortlog`: 18 / 8).
- **One commit per stage with a why-message** (e.g. "Resolve crossref inheritance (Stage 1.5)" explains the DBLP case it fixes). AI-written commits carry a `Co-Authored-By` trailer, so AI involvement is auditable in `git log`.
- **Prompts are versioned like code.** `doc/prompts/README.md` logs each significant prompt with the commit or PR it produced; task specs (`doc/tasks/`) and the design doc are prompts too and live beside the code.
- **Ownership through Git.** Partner modules were handed over as interfaces + `@Disabled` spec tests, implemented from the partner's own account.
- **Data hygiene.** The stakeholder's real corpus is git-ignored (`corpus.local/`, ADR-14).
- **Mistake we fixed:** a stacked PR (#3) merged into its base branch after that base had already merged, so `main` silently missed work. We now check `git rev-list main..branch` for every branch; PR #4 brought everything in.

4. Testing, refactoring and documentation

Practice	How we apply it
Tests per stage	23 test classes, 164 tests, none skipped. Normalizers use parameterised golden tables of real-world variants (DOI, LaTeX, author names).
End-to-end + real data	Synthetic two-paper corpus whose README lists each duplicate class and expected outcome; acceptance test: 12 records → 8 entries, 4 merges. Real IACR ePrint records: 0 false merges in 153 pairs.
Spec tests as contracts	Interfaces shipped with disabled tests stating exact behaviour; enabling them was the definition of done.
Architecture enforced	Maven enforcer bans Spring from the engine module (ADR-1), so logic cannot leak into controllers.
Refactoring with a safety net	Replaced the library LaTeX printer, reordered DOI normalization, added decision-ladder rules — each only with the full suite green before and after.
Documentation	Design doc (architecture, ADRs, test plan) in repo; Javadoc on every public type stating *why*; README with build/run; task specs; synthetic-corpus README; PR descriptions list deviations and known gaps.

Known gaps (stated, not hidden): blocking is all-pairs (fine for a few papers, not for 10k records); venue abbreviation dictionary and review UI not built; BibTeX/biber compile check not yet automated in CI.

Appendix — sample prompts (full log: doc/prompts/README.md)

A. Implement one stage under design constraints → PR #2

```
Implement my modules from design doc §6.6 (stages 1–6) on a branch, one commit per stage. Use the package
layout of §6.2 and the domain model of §5. Every stage needs unit tests before it is called done. Run the
full build after each commit and show me the test count. Do not touch Vamshi's modules (stages 7–9).
```

B. Spec-first hand-off → PR #3, then the partner's 8 commits

```
Do not write Vamshi's modules. Write interfaces that throw TODO(vamshi), one @Disabled test per class that
states the exact expected behaviour, and a task file listing the classes in dependency order. The build
must stay green.
```

C. Verify against real data → commit be07587

```
Get the stakeholder's real papers (public metadata only, stored in the git-ignored corpus.local/), run
`match` on them, and explain every MERGE or REVIEW decision.
```

D. Structured bug hunt → commits d6b9b9b, 929678a (PR #4)

```
Pull main. Build the integrated branch in a separate worktree. Probe: keys colliding across entries,
quotes in preambles, a line starting with @ inside a field, the real ePrint file. For each failure:
reproduce, write a failing regression test, fix, rerun the suite.
```

E. Template

```
Context: <design-doc section / files / current state to read first>
Task: <one module or one bug>
Constraints: <language, package, interfaces that must not change, what not to touch>
Acceptance: <tests that must pass, command, expected output>
Report: <diff summary, test counts, open questions>
```